Московский государственный университет имени М.В. Ломоносова
Факультет вычислительной математики и кибернетики
Кафедра суперкомпьютеров и квантовой информатики

Майстренко Марат Алексеевич

Параллельное выполнение алгоритма решения СЛАУ методом отражений

ОТЧЁТ ПО ВЫПОЛНЕНИЮ ЗАДАНИЯ №1

Москва, 2026

Оглавление

Алгоритм решения СЛАУ методом отражения 3

Алгоритм метода отражений 4

Постановка задачи 6

Реализация поставленной задачи 7

Тестовый интерфейс 9

Результаты 10

Заключение 13

Алгоритм решения СЛАУ методом отражения

Определение. Пусть дан единичный вектор **x** размерности n, (т.е. $||\mathbf{x}|| = 1$), тогда матрицей отражения называется $U(\mathbf{x}) = \mathbf{I} - 2\mathbf{x}\mathbf{x}^*$,

где * - транспонирование, норма - евклидова норма.

Лемма 1. Матрица $U(\mathbf{x})$ является ортогональной матрицей.

Лемма 2. Матрица $U(\mathbf{x})$ является симметричной матрицей.

Лемма 3. Матрица $U(\mathbf{x})$ имеет собственное значение -1 кратности 1, которому отвечает собственный вектор **x** и собственное значение 1 кратности n-1, которому отвечает собственное подпространство $\langle\mathbf{x}\rangle^{\perp} = \{\mathbf{y}: (\mathbf{x},\mathbf{y}) = 0\}$.

Лемма 4. Геометрический смысл преобразования, задаваемый матрицей $U(\mathbf{x})$: отражение относительно гиперплоскости $\langle\mathbf{x}\rangle^{\perp}$

Лемма 5. Пусть **e** - единичный вектор ($||\mathbf{e}||=1$), тогда для любого n-мерного вектора **y** существует такой единичный вектор **x**, что $U(\mathbf{x})\mathbf{y} = ||\mathbf{y}||\mathbf{e}$.

Лемма 6. Произведение матрицы отражения на вектор может быть вычислено за 2n-1 операцию сложения и 2n+1 операцию умножения.

Лемма 7. Произведение матрицы отражения (размера n x n) на матрицу размера n x m может быть вычислено за m(2n-1) операций умножения и m(2n+1) операций сложения.

Алгоритм метода отражений

Пусть требуется решить систему линейных уравнений **Ax**=**b**

Через $\mathbf{a}_1 = (a_{11},...,a_{1n})^*$ обозначим первый столбец матрицы **A**, согласно лемме 5 существует вектор $\mathbf{x}^{(1)}$, равный,

$$x^{(1)} = \pm \frac{a_1 - \|a_1\| e_1}{\|a_1 - \|a_1\| e_1\|}$$

такой что $U(\mathbf{x}^{(1)})\mathbf{a}_1 = \|\mathbf{a}_1\|\mathbf{e}_1$. Умножим на матрицу отражения левую и правую части уравнения **Ax**=**b**, получим уравнение $\mathbf{A}^{(1)}\mathbf{x}=\mathbf{b}^{(1)}$

$$A^{(1)} = U_1 A = U\left(x^{(1)}\right) A, \quad A^{(1)} = \begin{pmatrix} \|a_1\| & c_{12} & \dots & c_{1n} \\ 0 & a_{22}^{(1)} & \dots & a_{1n}^{(1)} \\ \dots & \dots & \ddots & \dots \\ 0 & a_{n2}^{(1)} & \dots & a_{nn}^{(1)} \end{pmatrix}, \quad b^{(1)} = U_1 b$$

Пусть сделан k-1 шаг алгоритма, и система приведена к виду $\mathbf{A}^{(k-1)}\mathbf{x} = \mathbf{b}^{(k-1)}$, где $\mathbf{A}^{(k-1)} = U_{k-1}U_{k-2}...U_1\mathbf{A}$, а $\mathbf{b}^{(k-1)} = U_{k-1}U_{k-2}...U_1\mathbf{b}$.

$$A^{(k-1)} = \begin{pmatrix} \|a_1\| & c_{12} & c_{13} & \dots & c_{1,k-1} & c_{1k} & \dots & c_{1n} \\ 0 & \|a_2\| & c_{23} & \dots & c_{2,k-1} & c_{2k} & \dots & c_{2n} \\ 0 & 0 & \|a_3\| & \dots & c_{3,k-1} & c_{3k} & \dots & c_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & \|a_{k-1}\| & c_{k-1,k} & \dots & c_{k-1,n} \\ 0 & 0 & 0 & \dots & 0 & a_{kk}^{(k-1)} & \dots & a_{kn}^{(k-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & 0 & a_{nk}^{(k-1)} & \dots & a_{nn}^{(k-1)} \end{pmatrix}, \quad U_i = \begin{pmatrix} I_{i-1} & 0 \\ 0 & U\left(x^{(i)}\right) \end{pmatrix}, \quad i=1,\dots,k-1$$

Обозначим через $\mathbf{a}_k = (a_{kk}, \dots, a_{nk})^*$. Тогда согласно лемме 5 существует вектор $\mathbf{x}^{(k)}$ (размерности n-k+1) равный

$$x^{(k)} = \pm \frac{a_k - \|a_k\| e_k}{\|a_k - \|a_k\| e_k\|}$$

такой, что $U(\mathbf{x}^{(k)})\mathbf{a}_k = \|\mathbf{a}_k\|\mathbf{e}_k$, где $\mathbf{e}_k = (1,0,\dots,0)$ (вектор размерности n-k+1)

Умножая матрицу U_k на левую и правую часть системы уравнений $\mathbf{A}^{(k-1)}\mathbf{x} = \mathbf{b}^{(k-1)}$, получим систему уравнений $\mathbf{A}^{(k)}\mathbf{x} = \mathbf{b}^{(k)}$, где $\mathbf{A}^{(k)} = U_k U_{k-1} U_{k-2} \cdots U_1 \mathbf{A}$, а $\mathbf{b}^{(k)} = U_k U_{k-1} U_{k-2} \cdots U_1 \mathbf{b}$.

$$A^{(k-1)} = \begin{pmatrix} \|a_1\| & c_{12} & c_{13} & \dots & c_{1,k-1} & c_{1k} & c_{1,k+1} & \dots & c_{1n} \\ 0 & \|a_2\| & c_{23} & \dots & c_{2,k-1} & c_{2k} & c_{2,k+1} & \dots & c_{2n} \\ 0 & 0 & \|a_3\| & \dots & c_{3,k-1} & c_{3k} & c_{3,k+1} & \dots & c_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & \|a_{k-1}\| & c_{k-1,k} & c_{k-1,k+1} & \dots & c_{k-1,n} \\ 0 & 0 & 0 & \dots & 0 & \|a_k\| & c_{k,k+1} & \dots & c_{k,n} \\ 0 & 0 & 0 & \dots & 0 & 0 & a^{(k)}_{k+1,k+1} & \dots & a^{(k)}_{k+1,n} \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & 0 & 0 & a^{(k)}_{n,k+1} & \dots & a^{(k)}_{nn} \end{pmatrix}, \quad U_k = \begin{pmatrix} I_{k-1} & 0 \\ 0 & U(x^{(k)}) \end{pmatrix}$$

Отметим, что при умножении матрицы вида U_k на матрицу вида $\mathbf{A}^{(k-1)}$ происходит умножение матрицы отражения на подматрицу матрицы $\mathbf{A}^{(k-1)}$ размера n-k+1 (остальная часть матрицы в преобразовании не участвует).

После n шагов процесса система примет вид $\mathbf{Rx} = \mathbf{y}$, где $\mathbf{R}$ - верхнетреугольная матрица. $R = \mathbf{A}^{(n)} = U_n U_{n-1} \cdots U_2 U_1 \mathbf{A}$, $\mathbf{y} = \mathbf{b}^{(n)} = U_n U_{n-1} \cdots U_2 U_1 \mathbf{b}$. Полученную систему можно решить используя обратный ход метода Гаусса.

Постановка задачи

Требуется написать параллельную программу с использованием технологии OpenMP для решения системы линейных уравнений **Ax**=**b** методом отражений.

Использование кластера **Polus** в этом задании **обязательно**.

Требования к программе:

1) Матрица **A** считается достаточно большой, но можно считать, что памяти одного узла достаточно для хранения всей матрицы.

2) Матрица **A** и вектор правой части **b** генерируется при помощи формулы, выдаваемой преподавателем.

3) Программа должна выводить норму невязки ||**Ax**-**b**||

4) В случае известного точного решения системы линейных уравнений программа должна выводить норму разницы между точным и полученным решением.

5) Программа должна выводить время, затраченное на приведение системы к верхнетреугольному виду (T_1), время, затраченное программой на обратный ход методом Гаусса (T_2), и общее время ($T_{all} = T_1+T_2$)

6) Программа должна корректно вычислять решение системы уравнений, при этом загрузка нитей должна быть максимально равномерной.

Реализация поставленной задачи

При разработке параллельной программы для решения СЛАУ методом отражений использована технология OpenMP. Основная вычислительная нагрузка приходится на прямой ход, где на каждом шаге k выполняется обновление всех столбцов матрицы, начиная с k-го. Внешний цикл по k является последовательным, внутренние операции по обновлению столбцов независимы и распараллеливаются.

Важной оптимизацией является переход на столбцовый формат хранения матрицы. В отличие от стандартного строчного формата (A[in + j]), *где индекс столбца меняется быстрее, в данной реализации используется формат A[i* + jn] с быстрым изменением индекса строки. Это обусловлено паттерном доступа к памяти в методе отражений: на каждом шаге алгоритма выполняется обновление столбцов матрицы при применении преобразования отражения. При столбцовом хранении элементы одного столбца располагаются в памяти последовательно, что обеспечивает эффективное использование кэш-памяти процессора и минимизацию кэш-промахов.

Вынесение директивы parallel за пределы внешнего цикла:

```
#pragma omp parallel shared(A,b,u,n,m,norm_x,alpha,u_norm)
{
    for(int k=0;k<n-1;k++)
    { ...
    }
}
```

Нити создаются один раз перед началом прямого хода и существуют на всех итерациях, что исключает накладные расходы на многократное создание нитей.

Параллельное обновление столбцов:

```
#pragma omp for schedule(dynamic,64)
for(int j=k;j<n;j++)
{ ...
}
```

Размер чанка, равный 64, выбран исходя из необходимости обеспечить равномерную загрузку нитей: на начальных шагах, когда столбцов много, нити получают достаточный объём работы, а на поздних шагах, когда столбцов остаётся мало, динамическое распределение позволяет избежать простоя части нитей.

Построение вектора отражения и обновление правой части:

```
#pragma omp single
{ ...
}

#pragma omp barrier

if(norm_x!=0.0)
{
    #pragma omp for schedule(dynamic,64)
    for(int j=k;j<n;j++)
    { ...
    }

    #pragma omp single
    { ...
    }
}

#pragma omp barrier
```

Секции single выполняются одной нитью. Барьеры после каждой секции гарантируют доступность вычисленных данных для всех нитей перед следующим этапом.

Привязка потоков к ядрам

```
#BSUB -n 32
#BSUB -W 00:30

export OMP_NUM_THREADS=32
export OMP_PROC_BIND=true
export OMP_PLACES=cores
```

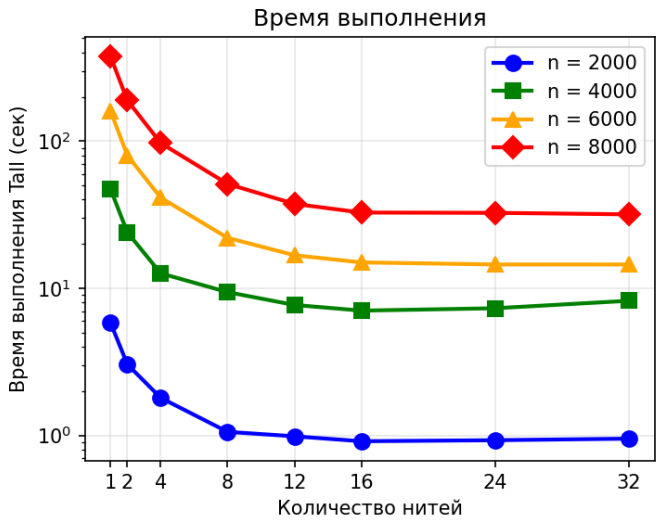
Запуск программы производится через систему LSF. В командном файле с помощью директивы #BSUB -n 32 запрашивается 32 вычислительных ядра на узле. Переменная OMP_NUM_THREADS=32 задаёт число создаваемых потоков OpenMP. Параметр OMP_PROC_BIND=true включает привязку потоков к вычислительным ресурсам, предотвращая их миграцию между ядрами во время выполнения. Параметр OMP_PLACES=cores указывает, что размещение потоков производится на уровне физических ядер процессора. При запуске создаётся 32 потока OpenMP, и каждый поток закрепляется за отдельным ядром.

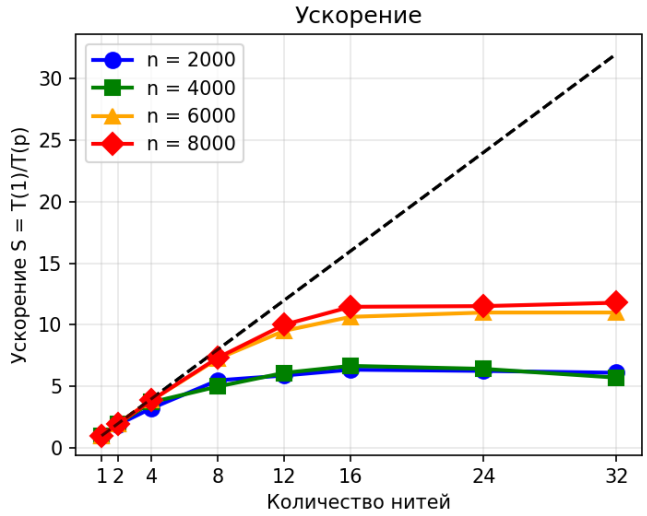
Тестовый интерфейс

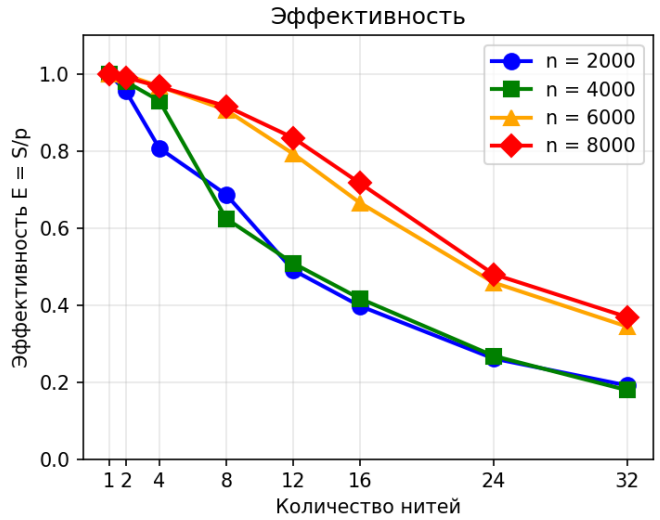
С помощью #define задается размер матрицы N. Количество нитей задается при запуске программы через аргументы командной строки.

Вывод: время приведения к треугольному виду T_1, время обратного хода T_2, общее время решения T_{all}, норма невязки ||Ax-b|| и норма разницы с точным решением ||x - x_true||. Результаты сохраняются в файл results.txt.

Результаты







Для всех четырёх размеров матрицы наблюдается устойчивое уменьшение времени решения при увеличении числа нитей вплоть до 16-24 нитей, после чего наступает насыщение.

Наибольший прирост производительности наблюдается при увеличении числа нитей от 1 до 8-16: для n=2000 время уменьшается с 5.82 до 0.68 секунд (ускорение около 8.5 раз), а для n=4000 - с 47.07 до 4.46 секунд. Аналогичная тенденция наблюдается и для больших размеров: при n=8000 время снижается с 375.13 до 32.27 секунд на 16 нитях. При увеличении числа нитей до 24-32 ускорение практически не растёт: для n=4000 время изменяется с 3.95 (24 нити) до 4.06 секунд (32 нити), а для n=2000 - с 0.64 до 0.66 секунд. Это связано с накладными расходами на синхронизацию.

При этом время обратного хода (T2) остаётся практически постоянным, поэтому основной вклад в ускорение даёт распараллеливание прямого хода (T1). Кроме того, на поздних шагах алгоритма число обрабатываемых столбцов уменьшается, что приводит к недозагрузке потоков и снижению эффективности при 24–32 нитях.

T_1 - время работы алгоритма метода отражений

T_2 - время работы обратного хода

$T_{all} = T_1 + T_2$

Threads	**T_1 (сек.)**	**T_2 (сек.)**	**T_{all} (сек.)**	**Невязка**	**Ошибка**
1	5.815521	0.003805	5.819326	4.726915e-12	7.395307e-12
2	3.040214	0.004003	3.044218	4.720109e-12	1.208592e-11
4	1.797935	0.003656	1.801591	4.867487e-12	2.775746e-11
8	1.055050	0.003680	1.058730	4.917903e-12	6.079592e-12
12	0.805991	0.003721	0.809712	4.811053e-12	1.271744e-11
16	0.683193	0.003768	0.686961	4.826509e-12	1.236510e-11
24	0.643558	0.003835	0.647393	4.910891e-12	1.114855e-11
32	0.656002	0.004115	0.660117	4.919554e-12	5.535169e-12

Таблица №1. Результаты эксперимента с n = 2000

Threads	**T_1 (сек.)**	**T_2 (сек.)**	**T_{all} (сек.)**	**Невязка**	**Ошибка**
1	47.038378	0.027634	47.066011	1.372036e-11	2.276137e-11
2	23.977256	0.028280	24.005536	1.316014e-11	3.308565e-11
4	12.611817	0.028418	12.640236	1.382769e-11	4.681241e-10
8	6.822568	0.026712	6.849281	1.397153e-11	8.891068e-12
12	5.217904	0.026002	5.243906	1.334354e-11	5.366576e-11
16	4.461133	0.026775	4.487908	1.379668e-11	1.178972e-11
24	3.950707	0.027222	3.977929	1.350148e-11	1.150265e-11
32	4.063016	0.027483	4.090499	1.329808e-11	1.886501e-11

Таблица №2. Результаты эксперимента с n = 4000

Threads	**T_1 (сек.)**	**T_2 (сек.)**	**T_{all} (сек.)**	**Невязка**	**Ошибка**
1	159.534008	0.168631	159.702640	1.912070e-11	9.026255e-11
2	79.841948	0.170409	80.012357	1.934851e-11	1.060582e-10
4	41.068589	0.170941	41.239531	1.967659e-11	7.765970e-11
8	21.852322	0.170638	22.022960	1.875028e-11	6.090991e-11
12	16.604642	0.171934	16.776576	1.917923e-11	4.415222e-11
16	14.825849	0.167921	14.993771	1.937193e-11	4.136150e-11
24	14.259471	0.259493	14.518964	1.892569e-11	1.405500e-11
32	14.243031	0.270143	14.513174	2.426961e-11	3.614136e-11

Таблица №3. Результаты эксперимента с n = 6000

Threads	**T_1 (сек.)**	**T_2 (сек.)**	**T_{all} (сек.)**	**Невязка**	**Ошибка**
1	374.690989	0.442232	375.133221	1.382769e-11	4.681241e-10
2	188.746401	0.441770	189.188172	1.397153e-11	8.891068e-11
4	96.490988	0.436734	96.927722	1.350148e-11	1.150265e-11
8	50.714186	0.444322	51.158508	1.329808e-11	1.886501e-11
12	37.003314	0.453189	37.456502	1.917923e-11	4.415222e-11
16	32.271921	0.444300	32.716221	1.937193e-11	4.136150e-11
24	31.835730	0.722582	32.558312	1.934851e-11	1.060582e-10
32	31.061420	0.736769	31.798189	1.967659e-11	7.765970e-11

Таблица №4. Результаты эксперимента с n = 8000

Заключение

В результате выполнения задания разработана параллельная программа на языке C с использованием OpenMP для решения СЛАУ методом отражений. Программа реализует прямой ход с вынесением директивы parallel за пределы внешнего цикла, динамическим распределением итераций (schedule(dynamic, 64)) и синхронизацией нитей с помощью барьеров, что обеспечивает равномерную загрузку нитей на всех этапах вычислений.

Проведены вычислительные эксперименты для матриц размером 2000, 4000, 6000 и 8000 с числом нитей от 1 до 32. Для n=2000 время решения сократилось с 5.82 до 0.64 секунд, для n=4000 - с 47.07 до 3.95 секунд, для n=6000 - с 159.70 до 14.52 секунд, для n=8000 - с 375.13 до 31.80 секунд. Наилучшее ускорение достигается при использовании 16-24 нитей, дальнейшее увеличение числа нитей даёт меньший прирост производительности из-за накладных расходов на синхронизацию.

Норма невязки во всех экспериментах составила 10^{-11} - 10^{-12}, что подтверждает корректность вычислений.

Анализ ускорения показывает, что для матрицы n=2000 максимальное ускорение составило 6.36 при 16 нитях (эффективность 0.40), для n=4000 - 6.67 при 16 нитях (эффективность 0.42), для n=6000 - 11.00 при 24 нитях (эффективность 0.46), для n=8000 - 11.80 при 32 нитях (эффективность 0.37). Рост эффективности с увеличением размера задачи подтверждает масштабируемость разработанного алгоритма.